A high level overview of machine learning performance evaluation for supervised learning problems

Anand Narasimhamurthy

Outline

- Measures for supervised learning problems
  - Evaluation Measures for Regression problems
  - Classification performance evaluation concepts Eg. Confusion matrix, Sensitivity, Specificity, ROC

- Methodology used for evaluation
  - Setting up and conducting experiments correctly : Eg. Train/test split
  - Bias variance trade off etc.

Commonly used error metrics in regression

Index (i)	Given y (y_i)	Predicted y $(\hat{y}_i)$	Absolute error $\lvert y_i - \hat{y}_i \rvert$	Squared error $(y_i - \hat{y}_i)^2$
1	y_1	$\hat{y}_1$	$\lvert y_1 - \hat{y}_1 \rvert$	$(y_1 - \hat{y}_1)^2$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
n	y_n	$\hat{y}_n$	$\lvert y_n - \hat{y}_n \rvert$	$(y_n - \hat{y}_n)^2$

Table illustrating computation of error metrics

- Mean Absolute Error (MAE)

$$MAE = \frac{1}{n}\sum_{i=1}^{n} |(y_i - \hat{y}_i)|$$

- Mean Squared Error (MSE)

$$MSE = \frac{1}{n}\sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

- Root Mean Squared Error (RMSE)

$$RMSE = \sqrt{MSE}$$

Commonly used error metrics in regression

- Mean Absolute Error (MAE)

$$MAE = \frac{1}{n}\sum_{i=1}^{n}|(y_i - \hat{y}_i)|$$

- Mean Squared Error (MSE)

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

- Root Mean Squared Error (RMSE)

$$RMSE = \sqrt{MSE}$$

- Mean Absolute Percent Error (MAPE)

$$MAPE = \frac{1}{n}\sum_{i=1}^{n}\left|\frac{(y_i - \hat{y}_i)}{y_i}\right|$$

Performance evaluation for Classification Models

Confusion matrix

Predicted labels →

Actual labels ↓

	1	**2**	**...**	**K**
1	a_{11}	a_{12}	...	a_{1K}
2	a_{21}	a_{22}	...	$a_{2,K}$
.	.	.	...	...
K	$a_{K,1}$	$a_{K,2}$	...	$a_{K,K}$

Accuracy =
Number of examples correctly predicted / Total number of examples

Confusion matrix for a classification problem with K classes.

Entry $a_{i,j}$ = Number of instances whose actual class label was i and predicted was j

Confusion matrix for a binary classification problem

Predicted labels

Actual labels

	1	0
1	TP	FN
0	FP	TN

Confusion Matrix for a binary classification problem (1 :Target class)

TP : Number of True Positives
FN : Number of False Negatives
FP : Number of False Positives
TN : Number of True Negatives

A few commonly used binary classification performance measures

Accuracy = (TP+TN)/N
Sensitivity = TP/ (TP+FN)
Specificity = TN/(TN+FP)
Precision = TP/ (TP + FP)

Note :

- Sensitivity is also called Recall
- False Positive Rate (FPR) = 1 - Specificity

Accuracy as performance metric

- Accuracy can often be a misleading metric, when one category occurs more often than other in the given data-set
  - For eg: Occurrence of cancer in general population is 0.4%
  - If a prediction system blindly marks everyone as “No cancer”, it will 99.6% accurate

A short review of performance metrics for classification

- Accuracy alone can be misleading, especially if there is a class imbalance.

- Hence, depending on the application better to report additional metrics Eg. For a two class problem with a target class, report either
  - Sensitivity and specificity or
  - Precision and Recall

Note : Recall is same as sensitivity but Precision is not same as specificity

- Most of the above measures can be derived from the **Confusion Matrix**.

- Another commonly used measure is the Kappa metric

Kappa Metric

- Kappa metric quantifies how accurate the prediction algorithm is when compared to a random prediction

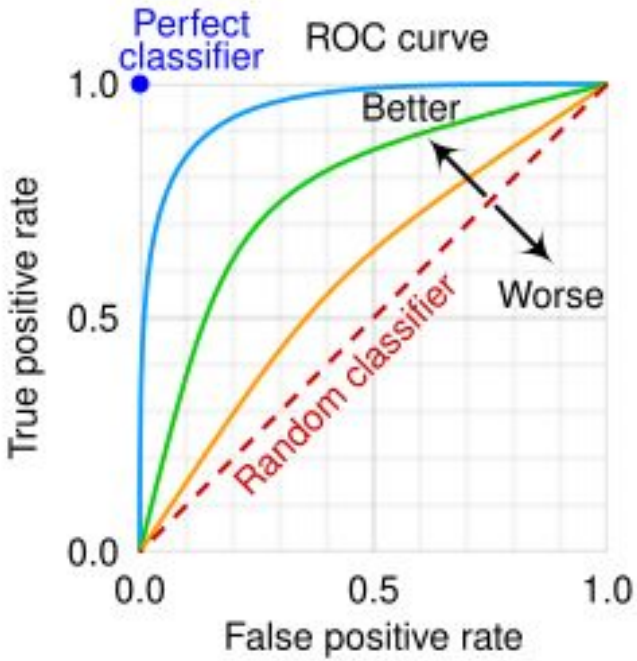
kappa = (totalAccuracy-randomAccuracy) / (1-randomAccuracy)

Kappa Value	
<0	No agreement
0-0.2	Slight
0.21 to 0.4	Fair
0.4 to 0.6	Moderate
0.6 to 0.8	Substantial
0.8 to 1	Almost Perfect

Receiver Operating Characteristics (ROC) Curves

ROC curve is suited for a binary classification problem, where there is a target class, it can be used to

- evaluate model performance for different parameter settings



- Compare different models : One possibility is to empirically compute ROC curves for each model and compute the Area Under the Curve (AUC)

Rough rule of thumb for Area Under the Curve (AUC) :

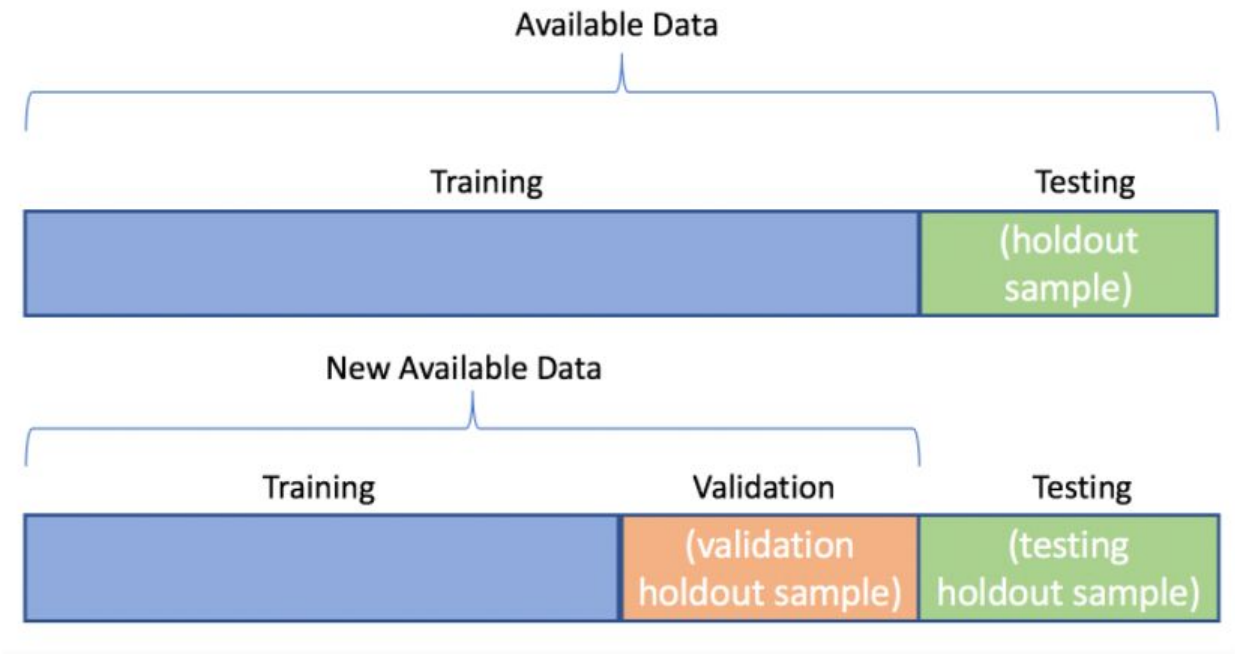
- 0.90 -1.0 = Excellent
- 0.80 – 0.90 = Good
- 0.70 – 0.80 = Fair
- 0.60 – 0.70 = Poor
- 0.50 – 0.60 = Fail
- <0.50 – You are better off doing a coin toss

Image reproduced from https://en.wikipedia.org/wiki/Receiver_operating_characteristic

Methodology for performance evaluation : A few broad objectives

- **Caveat :** Do not assess performance on the same data used for training the model
    - This would yield an overly optimistic estimate of the model performance
    - The model would likely **overfit** to the training data i.e. it would yield very good performance on training data but would perform a lot worse on new unseen test data
- The ultimate aim is to predict well on unseen samples

Split the available data into train/validation/test sets



Report performance metrics on the test set

Cross validation

A commonly used strategy for performance evaluation, especially in supervised learning.

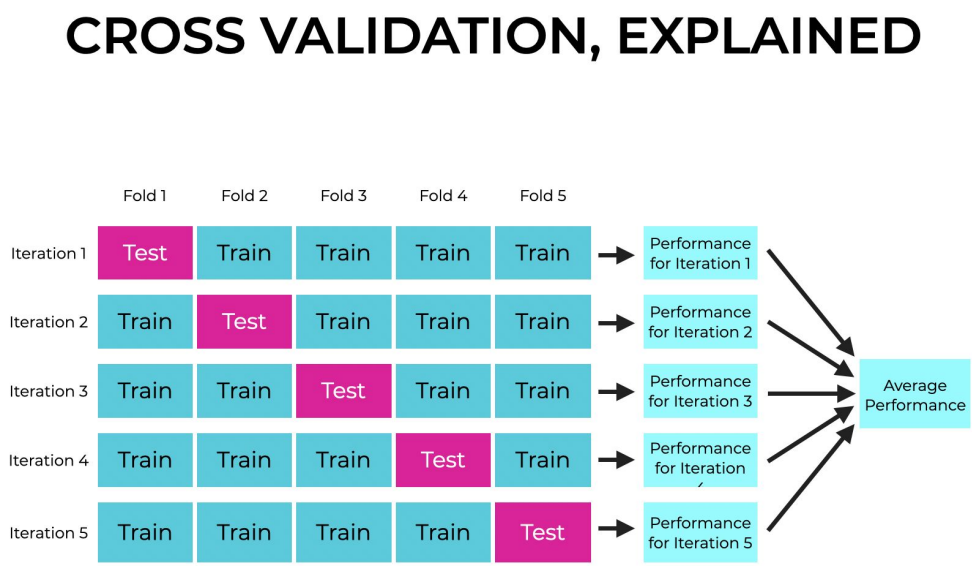


Image source : https://www.sharpsightlabs.com/blog/cross-validation-explained/

Cross validation

Common uses are for comparing performance of different models and/or parameter tuning.

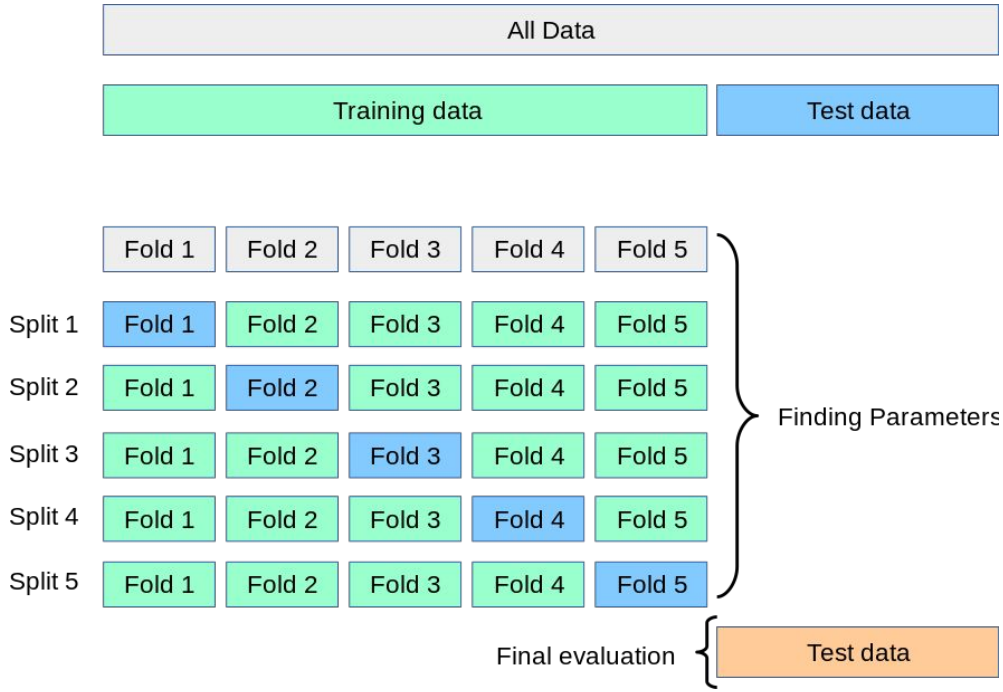
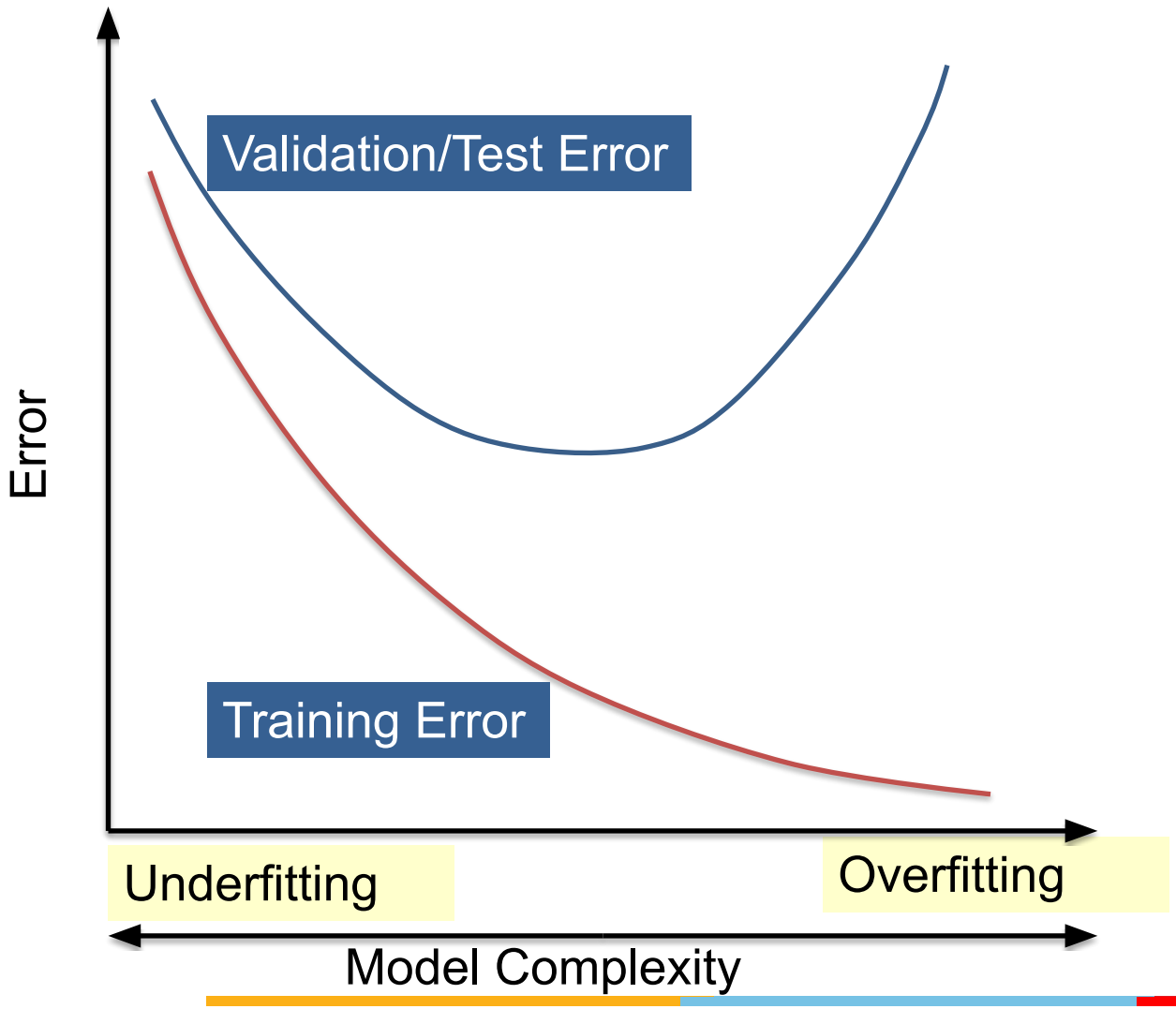


Image source : https://scikit-learn.org/1.5/modules/cross_validation.html

Underfitting vs Overfitting (Bias-Variance Tradeoff)